Topic 9 Chapter 7 Worksheet

In the worksheets for this class, you are going to create MySQL statement for each task described below.

You are required to take a screen shot of the SQL statements and the output for each task.

Paste the screenshot of your results under the numbered question.

Save your work and upload the document to the assignment folder by the due date in the Syllabus.

Use the Oracle tutorial database.

1. What is the average unit_price in order_items?

Next, go through the table and find all items whose unit_price is greater than the average. Group by item_id. Sort the output by item_id.

> Note: grouping by the item_id without also grouping by the price may generate a MySQL error: "Error Code: 1055. Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column."
>
> Issue this command to change the grouping rules, and then restart MySQL Workbench:
>
> ```
> SET @@global.sql_mode := REPLACE(@@global.sql_mode, 'ONLY_FULL_GROUP_BY', '');
> ```

13 rows returned

```
5      -- 1. What is the average unit_price in order_items?
6 •    SELECT AVG(unit_price)
7      FROM order_items;
8
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

AVG(unit_price)
872.235714

```sql
SELECT item_id, unit_price
FROM order_items
WHERE unit_price >
    (SELECT AVG(unit_price)
     FROM order_items)
GROUP BY item_id
ORDER BY item_id;
```

Result Grid | Filter Rows:

item_id	unit_price
1	1638.89
2	1431.99
3	1055.99
4	1029.99
5	1908.73
6	949.89
7	1449.99
8	948.99
9	1099.99
10	1029.99
11	1844.89
12	2200.00
13	1999.89

2. Join the above query with products to display the name of the item.

```
18      -- 2. Join the above query with products to display the name of the item.
19      SELECT oi.item_id, oi.unit_price, p.product_name
20      FROM order_items oi
21      JOIN products p USING(product_id)
22      WHERE oi.unit_price >
23          (SELECT AVG(unit_price)
24           FROM order_items)
25      GROUP BY oi.item_id
26      ORDER BY oi.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_id	unit_price	product_name
1	1638.89	Intel Xeon E5-2680 V3 (OEM/Tray)
2	1431.99	G.Skill Trident Z
3	1055.99	G.Skill Ripjaws 4 Series
4	1029.99	Intel Core i9-7900X
5	1908.73	Intel Xeon E5-2690 V3
6	949.89	PNY VCQK4200-PB
7	1449.99	Corsair Dominator Platinum
8	948.99	Supermicro X10SDV-8C-TLN4F
9	1099.99	Corsair Vengeance LPX
10	1029.99	Intel Core i9-7900X
11	1844.89	Intel Xeon E5-2683 V4 (OEM/Tray)
12	2200.00	Intel Xeon E5-2643 V2 (OEM/Tray)
13	1999.89	HP C2J95AT

3. List all the inventory that is stored in Mexico.

You do not know how many warehouses are in Mexico. Use a subquery to first find the warehouse id.

106 rows returned

```sql
-- 3. List all the inventory that is stored in Mexico.
SELECT i.warehouse_id, p.product_id, p.product_name
FROM inventories i
JOIN products p USING(product_id)
WHERE i.warehouse_id IN
    (SELECT w.warehouse_id
    FROM warehouses w
    JOIN locations l USING (location_id)
    JOIN countries c USING (country_id)
    WHERE c.country_name = 'Mexico');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouse_id	product_id	product_name
7	2	Intel Xeon E5-2697 V4
7	3	Corsair CB-9060011-WW
7	4	AMD 100-505989
7	5	PNY VCQK6000-PB
7	6	Zotac ZT-P10810A-10P
7	11	PNY VCQP5000-PB
7	12	Gigabyte GV-N108TAORUSX W-11GD
7	22	Seagate ST3000DM008
7	23	Western Digital WDS250G1B0A
7	24	Seagate ST2000DM006
7	25	Samsung MZ-75E250B/AM
7	26	Samsung MZ-75E500B/AM
7	33	ADATA ASU800SS-128GT-C
7	35	Corsair Dominator Platinum
7	36	Corsair Vengeance LPX
7	37	Corsair Dominator Platinum
7	38	Corsair Vengeance LPX

4. Rewrite the task above using a CTE.

```
39      -- 4. Rewrite the task above using a CTE.
40  •   WITH mexico_warehouses AS
41      (
42          SELECT w.warehouse_id
43          FROM warehouses w
44          JOIN locations l USING (location_id)
45          JOIN countries c USING (country_id)
46          WHERE c.country_name = 'Mexico'
47      )
48      SELECT i.warehouse_id, p.product_id, p.product_name
49      FROM inventories i
50      JOIN products p USING(product_id)
51      JOIN mexico_warehouses USING (warehouse_id);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouse_id	product_id	product_name
7	2	Intel Xeon E5-2697 V4
7	3	Corsair CB-9060011-WW
7	4	AMD 100-505989
7	5	PNY VCQK6000-PB
7	6	Zotac ZT-P10810A-10P
7	11	PNY VCQP5000-PB
7	12	Gigabyte GV-N108TAORUSX W-11GD
7	22	Seagate ST3000DM008
7	23	Western Digital WDS250G1B0A
7	24	Seagate ST2000DM006
7	25	Samsung MZ-75E250B/AM
7	26	Samsung MZ-75E500B/AM
7	33	ADATA ASU800SS-128GT-C
7	35	Corsair Dominator Platinum
7	36	Corsair Vengeance LPX
7	37	Corsair Dominator Platinum

5. Count the number of orders_items that are higher than the average order_item amount.

You will need to take the average of quantity * unit_price

```
53      -- 5. Count the number of orders_items that are higher than the average order_item amount
54      WITH amounts AS
55      (
56          SELECT quantity * unit_price AS item_amount
57          FROM order_items
58      )
59      SELECT COUNT(*) AS '# above average order items'
60      FROM amounts
61      WHERE item_amount > (SELECT AVG(item_amount) FROM amounts);
62
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

above average order items
252

6. Calculate the average order amount.

1. Use CTE and create a table that sums the quantity * unit_price for each order.

```
60      -- 6.1. Use CTE and create a table that sums the quantity * unit_price for each order.
61      WITH amounts AS
62      (
63          SELECT order_id, (quantity * unit_price) AS item_amount
64          FROM order_items
65      )
66      SELECT SUM(item_amount) AS total_order_amount
67      FROM amounts
68      GROUP BY order_id;
69
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_order_amount
1143716.8700
590302.9100
417049.2400
613735.0600
192101.1100
422361.9700
29267.8100
354128.5900
674744.8600
620962.9900
39687.5900
752198.6400
863963.8900
895138.6100
492688.4900
501733.7100

2. Use the CTE to calculate the average for the orders

```
74      -- 6.2. Use the CTE to calculate the average for the orders
75      WITH item_amounts AS
76      (
77          SELECT order_id, (quantity * unit_price) AS item_amount
78          FROM order_items
79      ),
80      order_totals AS
81      (
82          SELECT order_id, SUM(item_amount) AS total_order_amount
83          FROM item_amounts
84          GROUP BY order_id
85      )
86      SELECT AVG(total_order_amount) AS 'Average order'
87      FROM order_totals;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Average order
502305.64419048

7. How many customers have a credit limit lower than the average?

```
61      -- 7. How many customers have a credit limit lower than the average?
62      SELECT COUNT(*) AS 'Low credit limit'
63      FROM customers
64      WHERE credit_limit <
65          (SELECT AVG(credit_limit)
66          FROM customers);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Low credit limit
184